

🎤 AI 페어 프로그래밍의 한계를 넘어서

사전 기획부터 하네스 자가 치유 피드백 루프까지의 엔지니어링 여정

발표자: 프로젝트 테크 리더 (수석 엔지니어)

소속: 신한 딜리버리(shinhan-delivery) 프로젝트 팀

[Slide 1] 사전 기획 기반 AI 통합 개발 플로우

엔드투엔드 4단계 개발 파이프라인 (End-to-End Flow)

"이슈 분석과 기획부터 코드 구현, 자가 치유 피드백 루프까지 단일 연결 흐름으로 수행되는 개발 플로우를 설계하여 획기적인 개발 생산성을 달성했습니다."

- [1단계] 사전 기획/설계: MVP 기획서 작성 (요구사항) → 화면 설계서 작성 (UI/UX)
- [2단계] 이슈 발행/기획: 기획/화면설계서 기반 **GitHub Issue** 발행 → `/plan` AI 자동 분석 → 개발자 승인 **Checkpoint**
- [3단계] AI 연동 구현: 백엔드 비즈니스 로직 및 프론트엔드 화면 연동 연속 구현
- [4단계] 자가 치유 피드백: `./scripts/verify.sh` 자동 구동 → 실패 시 에러 로그 수집 → AI Auto-Fix 피드백 루프 → 마이크로 커밋(`commit`)

 **성과:** 보일러플레이트 작성 부담 대폭 단축 / 비즈니스 기능 개발 효율성 획기적 향상

[Slide 2] Phase 1: 화려한 성공 뒤의 5대 현실 고충 & 결함

⚡ 생산성의 덫: 팀원들이 겪은 4대 인적 고충과 5대 기술 결함

구분	팀원들이 실제로 겪은 인적/기술적 고충	실제 부작용 및 위험 타격
1. 도구/학습 과부하	AI 숙련도 부족 & 방대한 컨벤션 피로도	기술 스택과 AI도구를 동시 학습하는 정보 과부하, 핵심 수칙 구분 벅참
2. AI 맹목적 의존	'코딩 근육' 퇴화 & 메타인지(내가 뭘 모르는지) 상실	직접 고민 습관 사라짐, AI 없이는 코딩 힘들다는 무력감 & 메타인지 부재
3. UX / Thymeleaf	Thymeleaf SSR 대신 CSR AJAX 남발	Thymeleaf 사전 바인딩 외면하고 client AJAX만 호출해 초기 화면 텅 빈/깜빡임 발생
4. FE 파일 비분리	HTML/CSS/JS 분리 없이 인라인 결합	HTML 내 대용량 CSS/JS 섞여 캐싱 불가, 중복 스타일 파편화
5. 계층 위반	계층을 건너뛰고 내부 데이터 직접 접근	계층별 역할 분리 및 아키텍처 규칙 위반

🔧 [Slide 3] Phase 2: 아키텍처 재정립 & 자가 치유 피드백 루프

🔄 코드 작성 후 '자가 치유 피드백 루프 (Self-Healing Loop)' 구축

- 1. Thymeleaf SSR 사전 바인딩 (`th:each`): CSR AJAX 남용 제거로 초기 화면 텅 빈 및 깜빡이는 현상 완전 개선
- 2. HTML/CSS/JS 3-Tier 완전 분리 모듈화: 독립 static 파일 분리로 중복 코드 대량 제거 및 브라우저 캐싱 혜택 확보
- 3. 자가 치유 피드백 루프 구동:
 - `./scripts/verify.sh` 5대 품질 통제 게이트 구동 (Spotless + Checkstyle + ArchUnit + JaCoCo 60%+ + 400개 테스트)
 - 실패 시 에러 로그 수집 → AI에 피드백 → AI 수초 내 Auto-Fix → 100% 그린 빌드(0 Exit Code) 달성 시 마이크로 커밋 Execution

[Slide 4] Phase 3: 개발자가 사수해야 할 4대 핵심 수칙

 AI 시대, 개발자가 주도권을 쥐어야 할 4대 수칙

 AI는 "구현(How)"의 부사수일 뿐, "품질과 방향성(What & Architecture)"은 사람의 몫

1. [수칙 1] 선(先) 기획 및 사람이 직접 기술적 의사결정 주도 (**Planning-First**)
 - 개발 전 **MVP** 기획서와 화면 설계서를 선제 정의하고 핵심 방향성을 사람이 직접 지시
2. [수칙 2] 프론트엔드 **HTML/CSS/JS 3-Tier** 파일 분리 및 모듈화 (**Modularization**)
 - AI가 HTML 템플릿 내부에 CSS/JS를 인라인으로 섞지 못하도록 독립 파일 역할 분리 강제
3. [수칙 3] **Thymeleaf SSR vs CSR AJAX** 렌더링 역할 분담 (**Zero Flicker**)
 - 초기 DOM은 서버 사전 바인딩(SSR)으로 빠르게 띄우고, 진입 후 동적 연동만 클라이언트(CSR AJAX)
4. [수칙 4] 정적 분석 & 자동화된 하네스 통제망 구축 (**Harness First**)
 - `./scripts/verify.sh` 기반 0 Exit Code 자동 자가 치유 피드백 루프 회전

[Slide 5] Outro: KPT 회고 & 향후 극복 대책

 결론: AI 시대에 더욱 빛나는 수석 엔지니어의 디테일과 팀의 성장

KPT 회고 (Keep / Problem / Try)

- **Keep:** MVP기획/화면설계 선제 정의 + 자가 치유 피드백 루프로 매끄러운 화면 진입 & 결함 0개 달성
- **Problem:** AI 맹목적 의존으로 인한 '코딩 근육 퇴화', 메타인지 상실 및 컨벤션 과부하
- **Try (향후 극복 대책 3가지):**
 1. "Human-First Thinking (30분 고민 타임)": AI 사용 전 30분간 직접 고민하고 설계하기
 2. "팀 내 기초 CS & 아키텍처 스터디 정례화": '내가 뭘 모르는지' 인지하기 위한 자발적 스터디 병행
 3. "핵심 수칙 3대 숏컷 도식화": 벽찬 md 파일 대신 꼭 챙길 핵심 3대 수칙 위주로 시각화

Final Key Lesson:

"AI 도구는 개발자를 대체하는 것이 아니라, 사전 기획과 아키텍처를 고민하고 스스로 학습하는 개발자의 진짜 가치를 더욱 돋보이게 만든다."